

Министерство образования Республики Беларусь  
Учреждение образования  
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЕТ  
О ЛАБОРАТОРНОЙ РАБОТЕ №2  
на тему  
«СИСТЕМА УПРАВЛЕНИЯ ВЕРСИЯМИ»  
Вариант 21

Студент:

Л.С. Тарахович

Преподаватель:

Д.В. Куприянова

Минск 2026

## 1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить порядок работы с распределенной системой контроля версий Git, научиться выполнять базовые операции с локальным и удаленным репозиторием из интерфейса командной строки, а также получить практический навык управления ветками и совместной разработки проекта на платформе GitHub.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1 Установить консольный клиент Git и выполнить глобальную конфигурацию профиля пользователя с указанием реального имени и номера учебной группы, настроив аутентификацию исключительно по протоколу SSH.

- 2 Создать персональный форк учебного репозитория BSUIR-SDLC/SDLC на платформе GitHub и клонировать его на рабочую станцию для анализа структуры файлов и каталогов.

- 3 Создать ветку lab1 относительно ветки main, скопировать материалы лабораторной, проанализировать состояние рабочего дерева, настроить файл .gitignore, зафиксировать изменения коммитом и опубликовать ветку на GitHub.

- 4 Вернуться в ветку main, создать ветку ProjectTemplate, сформировать файл README.md с описанием разрабатываемого проекта, развернуть пустой каркас будущего семестрового приложения, настроить корневой файл .gitignore, сделать коммит и отправить ветку на сервер.

- 5 После проверки преподавателем выполнить слияние ветки ProjectTemplate в основную ветку main, отправить обновленную ветку на GitHub и провести анализ графа репозитория.

- 6 Ответить на контрольные вопросы и оформить отчет согласно действующему стандарту предприятия СТП БГУИР.

## **2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ**

### **2.1 Понятие системы контроля версий и её виды**

Система управления версиями (Version Control System, VCS) представляет собой программное обеспечение для фиксации, каталогизации и отслеживания изменений в исходном коде и сопутствующих материалах проекта. VCS предоставляет возможность просматривать историю правок, сравнивать различные состояния файлов, возвращаться к стабильным версиям и параллельно вести разработку изолированных компонентов. По архитектуре организации хранилища данных системы контроля версий разделяют на локальные, централизованные и распределенные. В централизованных системах (например, Apache Subversion или CVS) вся история изменений хранится на едином выделенном сервере, а разработчики получают только рабочие копии файлов без истории коммитов. В распределенных системах управления версиями (таких как Git или Mercurial) каждый участник команды хранит на своем компьютере полную копию репозитория со всей хронологией изменений проекта.

### **2.2 Особенности Git**

Фундаментальным отличием Git от традиционных систем контроля версий является способ фиксации изменений. Большинство традиционных VCS сохраняют информацию в виде списка изменений (дельт) для каждого отдельного файла во времени. Git рассматривает сохраняемые данные как набор моментальных снимков (снапшотов) миниатюрной файловой системы всего проекта на момент создания коммита. Для оптимизации дискового пространства неизменные файлы не дублируются, а сохраняются в виде жестких ссылок на ранее созданные неизменяемые блоки данных. Все объекты в репозитории идентифицируются с помощью криптографического хэширования по алгоритму SHA-1, что полностью исключает возможность незаметного повреждения или искажения истории. В архитектуре Git выделяются три логические области состояния файлов: рабочая директория (working directory), область подготовки к коммиту (staging area, или индекс) и локальный репозиторий (.git directory). Ветка в Git является лишь легковесным и подвижным указателем длиной в 41 байт, ссылающимся на конкретный хэш коммита. Благодаря этому операции создания новых веток, переключения между ними и слияния выполняются практически мгновенно и не требуют сетевого обмена.

### **2.3 Fork, Clone и авторизация по протоколу SSH**

При взаимодействии с веб-сервисами удаленных репозиториев (такими как GitHub или GitLab) широко применяется операция Fork. Операция Fork создает независимую копию стороннего репозитория в личной учетной записи

разработчика для безопасного внесения изменений без риска нарушить основной проект. Команда `git clone` загружает удаленный репозиторий по сети и развертывает его локальную копию с автоматической настройкой удаленного псевдонима `origin`. Для обеспечения криптографической защиты сетевого трафика и надежной аутентификации разработчика используется протокол Secure Shell (SSH). Протокол SSH базируется на паре асимметричных ключей: закрытом (`private key`), который хранится на компьютере пользователя в строжайшем секрете, и открытом (`public key`), размещаемом в настройках учетной записи GitHub. Такой способ авторизации исключает передачу учетных паролей по сети и предотвращает несанкционированный доступ к удаленному репозиторию.

## 3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

### 3.1 Установка и настройка Git

Инсталляция консольного клиента Git была выполнена на операционной системе семейства Windows в терминале PowerShell с помощью пакетного менеджера Winget. На рисунке 3.1 продемонстрирован вывод версии установленного клиента Git.

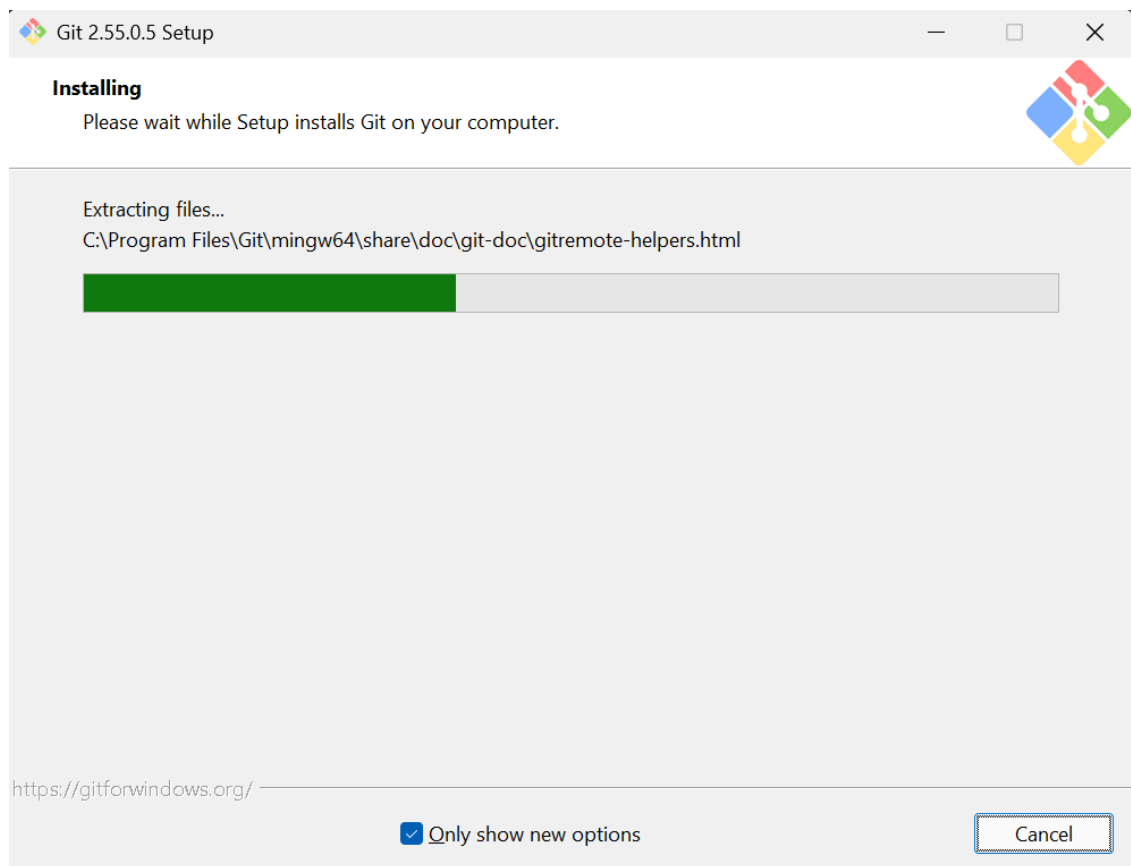


Рисунок 3.1 – Установка консольного клиента Git и проверка версии

После успешной установки была произведена глобальная настройка параметров профиля разработчика. С помощью команды `git config --global` были заданы параметры `user.name` с указанием имени и номера учебной группы, а также `user.email` с контактным почтовым адресом. Для настройки безопасного сетевого взаимодействия с удаленным сервером GitHub был сгенерирован асимметричный ключ по современному криптографическому алгоритму Ed25519. Сформированный публичный ключ из файла `id_ed25519.pub` был добавлен в раздел SSH-ключей профиля GitHub. Работоспособность и подлинность SSH-авторизации была протестирована командой `ssh -T git@github.com`. На рисунке 3.2 приведены результаты настройки конфигурации и успешной проверки аутентификации.

```
PS C:\Users\user> git config --global user.name "Тарахович Лев Сергеевич, гр. 450501"
PS C:\Users\user> git config --global user.email lev789tar@gmail.com
PS C:\Users\user> ssh -T git@github.com
Enter passphrase for key 'C:\Users\user/.ssh/id_ed25519':
Hi Levandovskii! You've successfully authenticated, but GitHub does not provide shell access.
```

Рисунок 3.2 – Настройка параметров конфигурации пользователя и проверка авторизации по SSH

## 3.2 Форк и клонирование репозитория

В веб-интерфейсе платформы GitHub была выполнена процедура Fork оригинального учебного репозитория BSUIR-SDLC/SDLC. В результате была сформирована персональная публичная копия проекта в личной учетной записи по адресу <https://github.com/Levandovskii/SDLC>. Созданный форк репозитория представлен на рисунке 3.3.

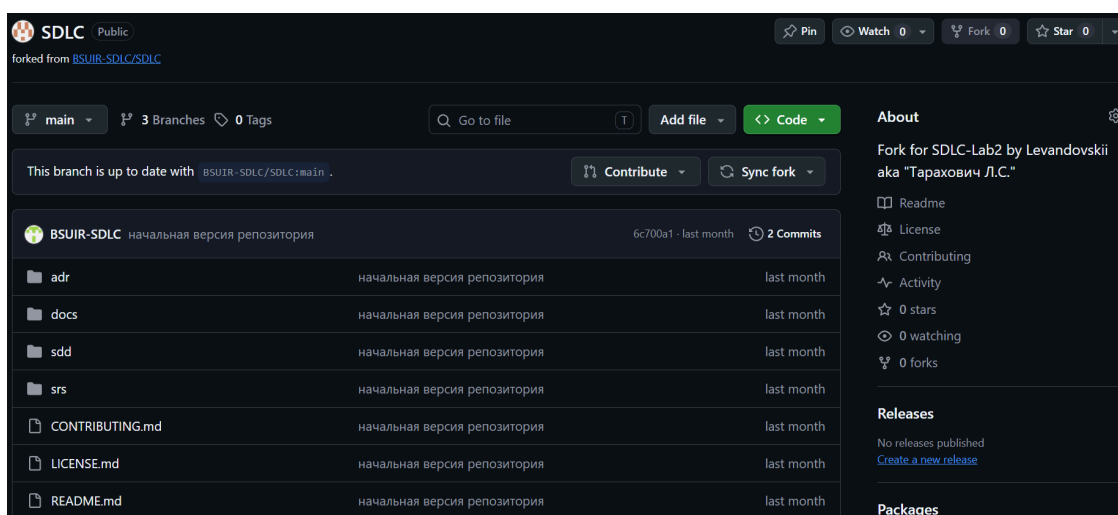


Рисунок 3.3 – Создание форка учебного репозитория в учетной записи GitHub

Далее созданный удаленный репозиторий был клонирован на локальный рабочий диск с использованием защищенного SSH-адреса. С помощью терминала выполнена синхронизация веток и первоначальное получение объектов репозитория. Процесс клонирования и синхронизации с удаленным источником origin проиллюстрирован на рисунке 3.4.

```
PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦППО\Labs\Lab02> git remote add origin git@github.com:Levandovskii/SDLC.git
PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦППО\Labs\Lab02> git pull origin main
Enter passphrase for key '/c/Users/user/.ssh/id_ed25519':
From github.com:Levandovskii/SDLC
* branch          main          -> FETCH_HEAD
```

Рисунок 3.4 – Клонирование репозитория по SSH и синхронизация с удаленным сервером

После перехода в корневой каталог проекта была изучена файловая структура шаблона, включающая подкаталоги с архитектурной документацией (adr, docs, sdd, srs) и теоретическими материалами.

### 3.3 Ветка lab1, перенос материалов ЛР №1 и настройка .gitignore

Для изолированного размещения наработок первой лабораторной работы от ветки main была создана новая тематическая ветка с именем lab1. Переключение на вновь созданную ветку выполнено с помощью консольной команды `git checkout -b lab1`. Фиксация создания ветки и переключения контекста представлена на рисунке 3.5.

```
PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> git branch lab1
PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> git switch lab1
Switched to branch 'lab1'
```

Рисунок 3.5 – Создание тематической ветки lab1 и переключение в неё

В рабочую директорию проекта был скопирован каталог первой лабораторной работы (Вариант 21: калькулятор лени «LazyCalculator»). Материалы работы включают файлы решения LazyCalculator.sln, проектный файл C# LazyCalculator.csproj, исходный код модели LazinessModel, контроллера LazinessController, интерфейсов подписки IObservable и IObserver, а также формы графического интерфейса Windows Forms. Операция создания структуры папок и копирования исходных кодов приведена на рисунке 3.6.

```
PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> mkdir lab1

Каталог: C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02

Mode                LastWriteTime         Length Name
----                -
d-----          9/29/2026   9:54 PM              lab1

PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> cp -r ../Lab01/LazyCalculator\ .\lab1\
```

Рисунок 3.6 – Создание директории и копирование материалов первой лабораторной работы

После копирования файлов был произведен анализ состояния рабочего дерева (working tree) посредством команды `git status`. Система контроля версий зафиксировала наличие новых неотслеживаемых каталогов и служебных файлов среды разработки. Результат выполнения команды анализа состояния каталога отображен на рисунке 3.7.

```

PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> git status
On branch lab1
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    lab1/

PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> ls lab1\

Каталог: C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02\lab1

Mode                LastWriteTime         Length Name
----                -
d-----          9/29/2026   9:55 PM                .idea
d-----          9/29/2026   9:55 PM                bin
d-----          9/29/2026   9:55 PM            Controller
d-----          9/29/2026   9:55 PM            Interface
d-----          9/29/2026   9:55 PM            Model
d-----          9/29/2026   9:55 PM              obj
d-----          9/29/2026   9:55 PM             report
d-----          9/29/2026   9:55 PM              View
-a-----          9/16/2026  12:11 PM           8496 .gitignore
-a-----          9/16/2026   2:44 AM           405 LazyCalculator.csproj
-a-----          9/15/2026   6:49 PM           278 LazyCalculator.csproj.user
-a-----          9/15/2026   6:49 PM           798 LazyCalculator.sln
-a-----          9/16/2026   2:44 AM           452 Program.cs

```

Рисунок 3.7 – Анализ состояния рабочего дерева и содержимого папки lab1

Для исключения попадания в репозиторий временных файлов компиляции, бинарных сборок и персональных настроек IDE был создан файл .gitignore. В конфигурационный файл были добавлены шаблоны исключения артефактов сборки .NET (папки bin/ и obj/), кэша сред разработки (.idea/, .vscode/), а также пользовательских параметров проектов (.user, .suo). Файлы первой лабораторной работы вместе с настроенным .gitignore были добавлены в индекс командой git add . и зафиксированы коммитом. После этого ветка lab1 была отправлена в удаленный репозиторий на GitHub с помощью команды git push -u origin lab1. Процесс индексации, фиксации коммита и отправки ветки на сервер приведен на рисунке 3.8.

```

PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦПНО\Labs\Lab02> git add -A
PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦПНО\Labs\Lab02> git status
On branch lab1
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitignore
    new file:   lab1/.gitignore
    new file:   lab1/Controller/LazinessController.cs
    new file:   lab1/Interface/Iobservable.cs
    new file:   lab1/Interface/Iobserver.cs
    new file:   lab1/LazyCalculator.csproj
    new file:   lab1/LazyCalculator.sln
    new file:   lab1/Model/LazinessModel.cs
    new file:   lab1/Program.cs
    new file:   lab1/View/InputDataForm.cs
    new file:   lab1/View/MainForm.cs

PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦПНО\Labs\Lab02> git commit -m "feat: Добавить предыдущую лабораторную"
[lab1 d3b4bbb] feat: Добавить предыдущую лабораторную
11 files changed, 927 insertions(+)
create mode 100644 .gitignore
create mode 100644 lab1/.gitignore
create mode 100644 lab1/Controller/LazinessController.cs
create mode 100644 lab1/Interface/Iobservable.cs
create mode 100644 lab1/Interface/Iobserver.cs
create mode 100644 lab1/LazyCalculator.csproj
create mode 100644 lab1/LazyCalculator.sln
create mode 100644 lab1/Model/LazinessModel.cs
create mode 100644 lab1/Program.cs
create mode 100644 lab1/View/InputDataForm.cs
create mode 100644 lab1/View/MainForm.cs
PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦПНО\Labs\Lab02> git push -u origin lab1
Enter passphrase for key '/c/Users/user/.ssh/id_ed25519':
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 16 threads
Compressing objects: 100% (15/15), done.
Writing objects: 100% (18/18), 9.64 KiB | 2.41 MiB/s, done.
Total 18 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'lab1' on GitHub by visiting:
remote:   https://github.com/Levandovskii/SDLC/pull/new/lab1
remote:
To github.com:Levandovskii/SDLC.git
 * [new branch]      lab1 -> lab1
branch 'lab1' set up to track 'origin/lab1'.

```

Рисунок 3.8 – Индексация, фиксация коммита и публикация ветки lab1 на GitHub

### 3.4 Создание шаблона проекта в ветке «ProjectTemplate»

Для подготовки базовой структуры сквозного курсового проекта был совершен переход обратно в основную ветку main, от которой была создана ветка ProjectTemplate. Операция переключения между ветками и создание ветки шаблона проекта показана на рисунке 3.9.

```

PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦПНО\Labs\Lab02> git switch main
Switched to branch 'main'

PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦПНО\Labs\Lab02> git switch -c ProjectTemplate
Switched to a new branch 'ProjectTemplate'

```

Рисунок 3.9 – Переход в основную ветку main и создание ветки ProjectTemplate

В корневом каталоге проекта был сформирован файл README.md. В файле README.md была отражена общая информация об авторе, наименование дисциплины («Жизненный цикл разработки программного обеспечения»),

а также описание архитектуры и бизнес-логики разрабатываемого калькулятора лени по Варианту 21. В репозитории был создан пустой проект приложения для последующей программной реализации задания. Также был актуализирован корневой файл .gitignore для фильтрации артефактов сборки и теоретических пособий. После этого с помощью команды git status было подтверждено отсутствие посторонних неотслеживаемых файлов. Состояние рабочего дерева после подготовки шаблона проиллюстрировано на рисунке 3.10.

```
PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> git add -A

PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> git status -s
A .gitignore
M README.md
A report/document/front-page-sdlc.typ
A report/document/report.typ
A report/main.typ
A src/SDLC/Form1.Designer.cs
A src/SDLC/Form1.cs
A src/SDLC/Program.cs
A src/SDLC/SDLC.csproj
A src/SDLC/SDLC.sln
```

Рисунок 3.10 – Проверка состояния рабочего дерева после подготовки README.md и структуры проекта

Все подготовленные изменения шаблона были зафиксированы коммитом и опубликованы в удаленном репозитории командой git push -u origin ProjectTemplate. Вывод консоли в процессе фиксации и отправки ветки в удаленный репозиторий представлен на рисунке 3.11.

```
PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> git commit -m "feat: Инициализация шаблона проекта и настройка gitignore"
[ProjectTemplate 51c445e] feat: Инициализация шаблона проекта и настройка gitignore
10 files changed, 226 insertions(+), 143 deletions(-)
create mode 100644 .gitignore
create mode 100644 report/document/front-page-sdlc.typ
create mode 100644 report/document/report.typ
create mode 100644 report/main.typ
create mode 100644 src/SDLC/Form1.Designer.cs
create mode 100644 src/SDLC/Form1.cs
create mode 100644 src/SDLC/Program.cs
create mode 100644 src/SDLC/SDLC.csproj
create mode 100644 src/SDLC/SDLC.sln

PS C:\Users\user\OneDrive\Desktop\КИ\5 сем\ЖЦРПО\Labs\Lab02> git push -u origin ProjectTemplate
Enter passphrase for key '/c/Users/user/.ssh/id_ed25519':
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 16 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (16/16), 3.68 KiB | 1.23 MiB/s, done.
Total 16 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'ProjectTemplate' on GitHub by visiting:
remote:   https://github.com/Levandovskii/SDLC/pull/new/ProjectTemplate
remote:
To github.com:Levandovskii/SDLC.git
 * [new branch]      ProjectTemplate -> ProjectTemplate
branch 'ProjectTemplate' set up to track 'origin/ProjectTemplate'.
```

Рисунок 3.11 – Фиксация коммита и публикация ветки ProjectTemplate в удаленный репозиторий

После согласования внесенных правок было выполнено слияние ветки ProjectTemplate в основную ветку main. Для этого был произведен переход в ветку main (`git switch main`), выполнена команда слияния `git merge ProjectTemplate` и отправка итогового состояния на сервер GitHub командой `git push origin main`.

## 4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

Ссылка на созданный удаленный репозиторий: <https://github.com/Levandovskii/SDLC>.

В ходе выполнения лабораторной работы была выстроена упорядоченная структура веток в удаленном репозитории на GitHub. Основная ветка `main` содержит актуальное стабильное состояние проекта, включая оформленный файл `README.md` с описанием индивидуального задания и корневой файл `.gitignore`. Тематическая ветка `lab1` хранит полный исходный код, структуру решения и материалы первой лабораторной работы (Вариант 21, приложение «LazyCalculator» на базе MVC с активной моделью). Ветка `ProjectTemplate` содержит историю поэтапного формирования каркаса проекта до момента его интеграции в ветку `main`. История репозитория не содержит артефактов сборки, объектных файлов, бинарных сборок и кэша сред разработки. На рисунке 4.1 продемонстрирован сетевой граф ветвления репозитория на платформе GitHub.

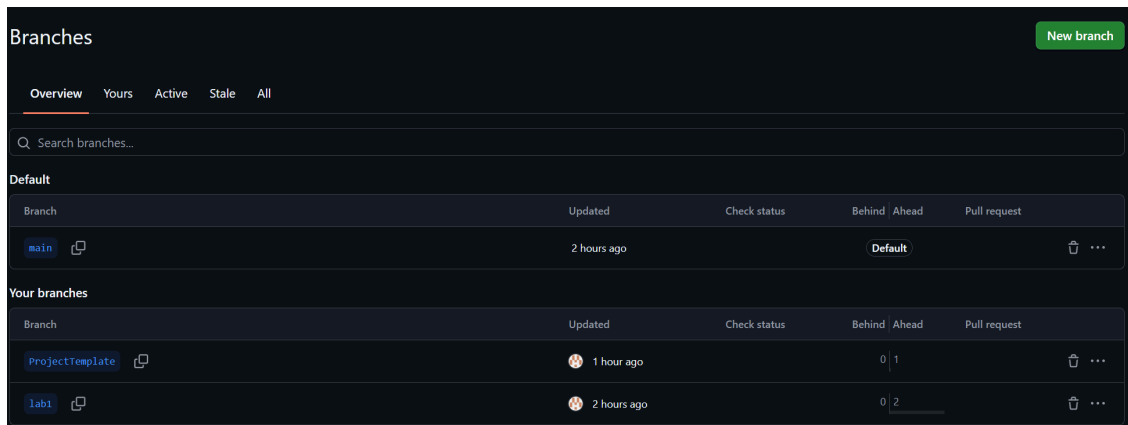


Рисунок 4.1 – Сетевой граф веток репозитория на платформе GitHub

## 5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

### Вопрос 1. В чем преимущество Git перед SVN?

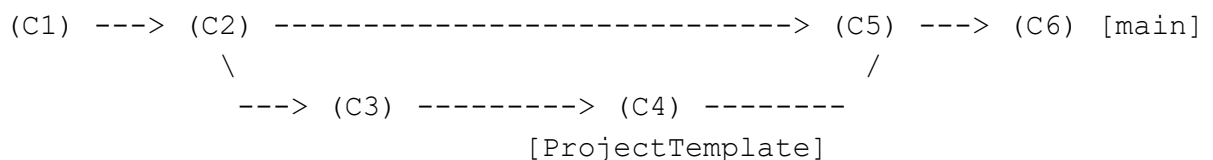
Главное преимущество Git перед централизованной системой SVN обусловлено его распределенной топологией. В Git каждый участник разработки располагает полной локальной копией всей истории проекта. Это позволяет выполнять практически все базовые операции (просмотр истории, коммиты, переключение веток, откат правок) локально без сетевого соединения с сервером. В отличие от SVN, где создание ветки требует физического копирования дерева файлов на сервере, ветвление в Git происходит мгновенно путем создания 41-байтового указателя на коммит. Кроме того, Git гарантирует целостность каждого файла и снимка состояния за счет криптографического контроля на основе хэш-сумм SHA-1.

### Вопрос 2. В чем преимущество распределённых репозиторий?

Распределенная архитектура исключает наличие единой точки отказа (Single Point of Failure), свойственной централизованным системам. В случае сбоя, недоступности или повреждения данных на центральном сервере репозиторий может быть целиком восстановлен из локальной копии любого разработчика. Разработчики получают возможность непрерывно продолжать работу над проектом и делать коммиты в автономном режиме без подключения к интернету. Также распределенная модель обеспечивает гибкость в построении процессов разработки, поддерживая топологии с форками репозиторий и системой рецензирования через Pull Request.

### Вопрос 3. Нарисуйте граф типичной работы с веткой master в системе контроля версий Git.

Типичный процесс работы с основной веткой main (ранее master) строится вокруг изоляции экспериментальной разработки в независимых ветках функционала (feature branches) с последующим слиянием обратно в стабильную ветку. Схематический граф типичного жизненного цикла ветвления приведен на рисунке 5.1.



Листинг 5.1 – Граф типичной работы с основной веткой репозитория

На приведенной схеме от коммита C2 основной ветки main отделяется тематическая ветка ProjectTemplate. В изолированной ветке выполняются локальные фиксации правок C3 и C4. После завершения и верификации работы

изменения интегрируются обратно в ветку `main` посредством создания коммита слияния `C5`.

**Вопрос 4. Назовите наиболее часто используемые команды командной строки.**

Для организации повседневного рабочего процесса в консоли `Git` применяется базовый набор команд:

- `git init` – инициализация нового чистого репозитория в текущей локальной папке.

- `git clone <url>` – копирование существующего удаленного репозитория на локальный компьютер.

- `git status` – вывод текущего состояния файлов в рабочем дереве и в индексе подготовки к коммиту.

- `git add <file>` – добавление файлов из рабочей директории в область индекса (staging area).

- `git commit -m "<msg>"` – создание снимка состояния проекта с фиксацией индексированных изменений.

- `git branch` – вывод списка существующих веток, создание новой ветки или удаление ветки.

- `git checkout` (или `git switch`) – переключение между существующими ветками или восстановление файлов.

- `git merge <branch>` – слияние изменений из указанной ветки в текущую активную ветку.

- `git fetch` – загрузка новых коммитов и ссылок с удаленного сервера без изменения рабочих файлов.

- `git pull` – скачивание изменений с удаленного сервера с автоматическим слиянием в текущую локальную ветку.

- `git push` – отправка зафиксированных локальных коммитов в удаленный репозиторий.

- `git log` – отображение хронологической истории коммитов с информацией об авторах и хэшах.

**Вопрос 5. Что нужно делать, если кто-то успел внести в удалённый репозиторий изменения?**

Если при выполнении команды `git push` сервер отклоняет отправку коммитов из-за наличия чужих правок, необходимо предварительно синхронизировать локальную ветку. В первую очередь выполняется команда `git fetch` для безопасного скачивания новых объектов и перемещения удаленного указателя `origin/main`. Затем выполняется интеграция обновлений с помощью команды `git merge origin/main` либо с помощью перемещения коммитов командой `git rebase origin/main`. Также эти два действия можно совместить выполнением команды `git pull`. Если в процессе слияния возникают конфликты в несовпадающих

строках кода, Git останавливает операцию и помечает конфликтные блоки специальными маркерами. Разработчик обязан вручную разрешить конфликты в редакторе, сохранить файлы, добавить их в индекс командой `git add`, завершить коммит и повторно выполнить команду `git push`.

## ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были освоены теоретические основы и практические навыки взаимодействия с распределенной системой управления версиями Git и сервисом GitHub. Были закреплены знания о внутренней архитектуре Git, включая различия между рабочим деревом, областью подготовки к коммиту (индексом) и историей репозитория. На практике выполнена настройка глобальной конфигурации пользователя и организована безопасная аутентификация по протоколу SSH с использованием пары асимметричных ключей алгоритма Ed25519. Отработан полный цикл ветвления: создание персонального форка учебного репозитория, клонирование на локальную станцию, перенос материалов первой лабораторной работы в изолированную ветку lab1 и формирование чистого шаблона проекта .NET Windows Forms в ветке ProjectTemplate. Особое внимание было уделено поддержанию чистоты истории репозитория с помощью создания и настройки файлов .gitignore, что предотвратило попадание артефактов сборки .NET и кэша сред разработки в удаленное хранилище. Все поставленные цели достигнуты, а задачи лабораторной работы выполнены в полном объеме.

# ПРИЛОЖЕНИЕ А

## (обязательное)

### Листинг кода

#### Файл README

# Учебный проект по ЖЦРПО

## Общая информация

\* \*\*Автор:\*\* Тарахович Лев Сергеевич

\* \*\*Группа:\*\* 450501

\* \*\*Дисциплина:\*\* Жизненный цикл разработки программного обеспечения

## Описание проекта

Проект представляет собой Windows Forms приложение для оценки продуктивности пользователя и расчета уровня прокрастинации.

### Основные функции:

\* \*\*Ввод данных через диалоговое окно:\*\* пользователь указывает количество дел, запланированных на день, и количество фактически выполненных.

\* \*\*Расчёт показателей:\*\* вычисление коэффициента лени, определение уровня прокрастинации и вывод мотивационной или саркастической фразы.

\* \*\*Сохранение состояния:\*\* запоминание последних введенных значений и их автоматическое восстановление при повторном открытии диалога.

\* \*\*Валидация:\*\* проверка корректности введенных числовых данных и вывод сообщений при ошибках ввода.

\* \*\*Архитектура:\*\* реализация логики на основе паттерна MVC (Model-View-Controller) с активной моделью.

#### ProjectTemplate .gitignore

.idea/

theoretics/

bin/

obj/

\*.user